

HFT Project — Change Log

Live-trading stack & hardware market maker • July 2026

What was built this session: the full software trading system, real broker execution, four strategies, and a timing-closed *hardware* market maker.

1. Overview

The prior work was a timing-closed FPGA tick-to-trade *signal* pipeline (205 ns) plus a software mirror. This session built the entire layer *around* that: a multi-symbol live-trading stack with real broker execution, a disciplined pre-trade decision engine, four trading strategies, a live web monitor, and — on the hardware side — a new, separately timing-closed **market-maker accelerator**. None of the existing RTL signal core changed; everything is either a software overlay or a new, independently verified hardware block. All work is committed locally on branch `feat/live-trading-and-hw-market-maker` (79 files); the AWS HDK vendor clone and build artifacts are excluded.

2. Software trading stack (`host/`)

- **Multi-symbol live feed** — trades a named universe (Magnificent 7) or any basket concurrently, one strategy/book/position per symbol.
- **Pre-trade gauntlet** (`hft_logic.py`) — every signal must pass, in order: fair value (microprice) → edge vs cost → volatility / liquidity / staleness → inventory skew → portfolio exposure + kill switch. Each veto is counted (a funnel).
- **Portfolio & risk** (`portfolio.py`) — cross-symbol positions, realized/unrealized PnL, net/gross exposure, per-symbol drawdown, fees, and a daily-loss/drawdown **kill switch**.
- **Execution safety** — per-position stop-loss / take-profit, max-hold auto-flatten, cooldown throttle, auto-flatten on stop and on kill-switch, and a **Flatten** / **Pause** / **Stop** control from the monitor page.

3. Real broker execution (Alpaca / IBKR)

Live/paper trading now runs on *real fills*, not assumptions:

- Orders are booked from the broker's actual **trade-update** / **execDetails** events — real quantity, price, and fee — for both Alpaca and IBKR.
- The portfolio is **reconciled to broker truth** at startup and periodically (real positions + average entry), so equity/exposure are broker-accurate.
- Real **slippage** (fill vs decision price) and real **network RTT** (order submit→ack) are measured, not modelled.

A live AAPL run confirmed the plumbing end-to-end (real fills, positions, PnL, slippage) and surfaced the honest finding below.

4. Strategies & backtest

Four strategies, compared on the same streams (`backtest.py`, 1 bps fee):

Strategy	Mean net PnL	Behaviour
Taker (crosses spread)	−\$13,139	pays the spread → bleeds
Market maker (heuristic)	+\$620	earns spread, inventory-skewed
Avellaneda–Stoikov (optimal)	+\$266	reservation price + optimal spread
Momentum taker	(loses live)	buys up-moves → adverse selection

Honest takeaways. (1) Crossing the spread loses; *earning* it (market making) pays. (2) The imbalance signal has *no* short-horizon edge on real AAPL (measured information coefficient was negative) — so the taker's edge is latency, not alpha. (3) The A–S model needed real care: an uncalibrated volatility term first blew up (−\$1.7M) until **gamma** was scaled to tick units and quotes were bounded; and a too-tight “optimal” spread eats adverse selection on a noisy feed (a wider floor turns it positive).

5. Live web monitor (`reports/live.html`)

A self-contained control panel served by the feed itself:

- **Speed hero** — SW compute latency vs the FPGA’s fixed 205 ns, a cumulative “time saved” counter, a live latency histogram + percentiles.
- **P&L / positions** — equity + realized-only overlay, per-symbol PnL, **shares bought / sold, open positions**, exposure vs cap, risk-vs-loss.
- **Price & spread chart**, pre-trade funnel, recent trades (with per-trade compute time + slippage), alerts feed, live log.
- **Controls** — start/stop/pause, switch strategy, tune config, add/remove symbol, flatten — all from the page; dark terminal theme; keyboard shortcuts.

6. Hardware market maker (new RTL)

The one piece that genuinely warranted new RTL + synthesis:

- `strategy_market_maker.sv` — microprice fair value + inventory skew + half-spread → two-sided quotes, with position-cap suppression.
- `pipe_divu.sv` — a 64-stage pipelined divider. A market maker *must* divide (it produces a price), and a single-cycle divide failed timing badly (WNS −39.3 ns, the 387-level division was the critical path). Pipelining it fixed it.
- `market_maker_top.sv` — AXI-Lite CSR accelerator + latency counter.

Metric (Vivado OOC, xcu200 @ 200 MHz)	single-cycle	pipelined (final)
Worst negative slack (WNS)	−39.3 ns (fail)	+ 1.004 ns (met)
Failing endpoints	64	0
CLB LUTs	2,733	9,237 (0.78%)
DSPs	8	8
Compute latency	—	66 cycles (330 ns)

Verified bit-exact against a golden model (500/500 and 40/40 cocotb checks). An AFI-buildable F2 Custom Logic wrapper (`f2/cl_market_maker/`) mirrors the proven `cl_tick_to_trade` — ready to bake into an AFI once the AWS F-instance quota clears.

7. Verification & status

- One-command regression (`run_all_tests.sh`): **6/6 green** throughout.
- Software signal core still equivalent to the RTL (0 mismatches, 16,361 decisions).
- Both new RTL blocks verified vs golden and timing-closed.
- AWS: only F2 remains (F1 end-of-life); the on-silicon run is blocked solely by the F-instance vCPU quota, not by the design. Everything else is done and local.